

Artificial Intelligence Exam & Lab Preparation

Uninformed Search, Informed Search, A*, Local Search,
Gradient Descent, Genetic Algorithm, Machine Learning
and Neural Networks

Prepared by: Ashikuzzaman Ashik

Department of Computer Science and Engineering
Pabna University of Science and Technology

Contents

1	Uninformed Search	3
1.1	State versus Node	3
1.2	Search Evaluation Criteria	3
1.3	Breadth-First Search (BFS)	3
1.4	Depth-First Search (DFS)	5
1.5	BFS versus DFS	6
1.6	Uniform Cost Search (UCS)	7
2	Informed / Heuristic Search	8
2.1	Greedy Best-First Search	8
2.2	A* Search	10
3	Hill Climbing – Local Search and Optimization	13
3.1	Hill-Climbing Rule	13
3.2	Important Problems	13
3.3	Mathematical Example	13
3.4	Random-Restart Hill Climbing	15
4	Gradient Descent	15
4.1	Mathematical Example	15
4.2	Gradient Descent – Lab Example	16
5	Genetic Algorithm	17
5.1	Fitness-proportionate Selection	17
5.2	Crossover Example	17
5.3	Mutation Example	18
6	Machine Learning	18
6.1	Classification	19
6.2	Important Classification Algorithms from the Lab	19
6.3	How a Machine Learning Model Works	20
6.4	Regression	21
6.5	Confusion Matrix	21
6.6	Classification Metrics	21
6.7	Classification Numerical Problem	22
6.8	Regression Metrics	23
7	Neural Network – Lab Sheet Style	24
7.1	Forward Pass	24
7.2	Loss Function	24
7.3	Backpropagation	24
7.4	Complete Numerical Example from the Lab	24
8	Lab Test #1 – Complete Solution	25
9	AI Lab Assignments – Important Mathematical Problems	28
9.1	Assignment 1 – A* Search: Route Finding	28
9.2	Assignment 2 – A* Search: Duplicate Paths and Consistency	31
9.3	Assignment 3 – Hill Climbing: Local Maximum	32
9.4	Assignment 4 – Simulated Annealing	33

9.5	Assignment 5 – Gradient Descent: One Variable	34
9.6	Assignment 6 – Gradient Descent for Linear Regression	36
9.7	Assignment 7 – Genetic Algorithm: Binary Maximization	37
9.8	Assignment 8 – Genetic Algorithm: Selection Probabilities	38
9.9	Assignment 9 – MLP Forward Pass with ReLU	39
9.10	Assignment 10 – MLP Backward Pass with ReLU	41
9.11	Additional MLP Worked Example – $y = 2x + 1$	42
10	Quick Formula Sheet	44

1 Uninformed Search

Uninformed (blind) search does not use additional problem-specific heuristic information. It relies on the structure of the problem.

Important examples:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform Cost Search (UCS)
- Depth-Limited Search
- Iterative Deepening Search (IDS)
- Bidirectional Search

The lecture material explicitly lists BFS, uniform-cost, DFS, iterative deepening, bidirectional, and branch-and-bound under uninformed/blind search.

1.1 State versus Node

A **state** represents a physical configuration of the problem. A **node** is a search data structure containing information such as state, parent, action, path cost, and depth.

1.2 Search Evaluation Criteria

- **Completeness:** Will it find a solution if one exists?
- **Optimality:** Will it find a least-cost solution?
- **Time complexity:** Number of generated nodes.
- **Space complexity:** Maximum number of nodes stored.

1.3 Breadth-First Search (BFS)

Core idea: Expand the shallowest unexpanded node first.

BFS uses a FIFO queue:

First In, First Out

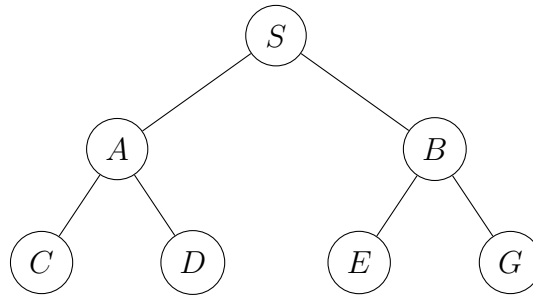
So BFS explores the search tree level by level.

Properties

Completeness	Yes
Optimality	Yes, when path cost is a non-decreasing function of depth
Time	$O(b^d)$ in the standard simplified form
Space	$O(b^d)$ in the standard simplified form
Main drawback	High/exponential memory requirement

Exam-style Problem

Question: Starting from S , find G using BFS. Show the queue after each expansion.

**Solution:**

Initial:

$$Q = [S]$$

Expand S :

$$Q = [A, B]$$

Expand A :

$$Q = [B, C, D]$$

Expand B :

$$Q = [C, D, E, G]$$

Expand C :

$$Q = [D, E, G]$$

Expand D :

$$Q = [E, G]$$

Expand E :

$$Q = [G]$$

Select G :

Goal found

Therefore the BFS path is

$S \rightarrow B \rightarrow G$

Python Code

```

from collections import deque

graph = {
    'S': ['A', 'B'],
    'A': ['C', 'D'],
    'B': ['E', 'G'],
    'C': [], 'D': [], 'E': [], 'G': []
}

def bfs(start, goal):
    q = deque([(start, [start])])
    visited = set([start])

    while q:

```

```

node, path = q.popleft()
print("Expand:", node, "Queue:", list(q))

if node == goal:
    return path

for child in graph[node]:
    if child not in visited:
        visited.add(child)
        q.append((child, path + [child]))

print("BFS Path:", bfs('S', 'G'))

```

1.4 Depth-First Search (DFS)

Core idea: Expand the deepest unexpanded node first.

DFS uses a LIFO structure:

Last In, First Out

The important memory idea from the lecture is that DFS keeps space mainly for the current tree path plus the necessary unexplored alternatives, rather than storing an entire breadth frontier.

Properties

Completeness	No for unbounded-depth/infinite paths
Optimality	No
Time	$O(b^m)$
Space	$O(bm)$
Main advantage	Much lower memory use than BFS

Exam-style Problem

Use the same tree and start at S :

$$S \rightarrow A, B; \quad A \rightarrow C, D; \quad B \rightarrow E, G$$

Assume children are considered left-to-right.

Solution:

Start:

$$Stack = [S]$$

Expand S :

$$Stack = [A, B]$$

DFS takes the deepest/most recently inserted branch first:

$$A$$

Then:

$$Stack = [C, D, B]$$

Next:

$$D$$

Then backtrack and continue according to the stack ordering.

For this tree, the goal can be reached through

$$\boxed{S \rightarrow B \rightarrow G}$$

depending on the exact child insertion convention used by the implementation. Therefore, in an exam, always show the stack order.

Python Code

```
graph = {
    'S': ['A', 'B'],
    'A': ['C', 'D'],
    'B': ['E', 'G'],
    'C': [], 'D': [], 'E': [], 'G': []
}

def dfs(start, goal):
    stack = [(start, [start])]
    visited = set()

    while stack:
        node, path = stack.pop()
        print("Expand:", node, "Stack:", [x[0] for x in stack])

        if node == goal:
            return path

        if node not in visited:
            visited.add(node)

            # Reverse so leftmost child is processed first
            for child in reversed(graph[node]):
                if child not in visited:
                    stack.append((child, path + [child]))

    print("DFS Path:", dfs('S', 'G'))
```

1.5 BFS versus DFS

Feature	BFS	DFS
Strategy	Shallowest first	Deepest first
Data structure	FIFO queue	LIFO stack
Complete	Yes under standard finite conditions	No for unbounded depth
Optimal	Under unit/equal-step-cost condition	No
Time	$O(b^d)$	$O(b^m)$
Space	$O(b^d)$	$O(bm)$
Memory	High	Low

Important: Typical numerical question: Given b , d , and m , calculate and compare the time and space complexity of BFS and DFS.

Numerical Example

Given:

$$b = 3, \quad d = 4, \quad m = 7$$

BFS:

$$Time = O(3^4) = O(81)$$

$$Space = O(3^4) = O(81)$$

DFS:

$$Time = O(3^7) = O(2187)$$

$$Space = O(3 \times 7) = O(21)$$

Thus DFS uses much less space for this example, while BFS has a smaller worst-case exponent when $d \ll m$.

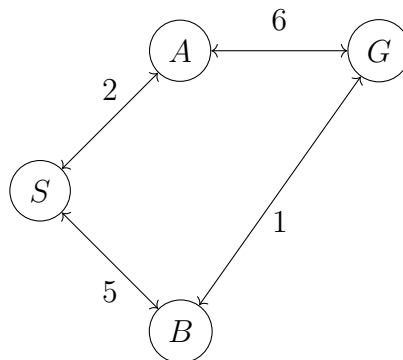
1.6 Uniform Cost Search (UCS)

UCS expands the frontier node having minimum accumulated path cost:

$$g(n)$$

$$g(n) = \text{cost from start to } n$$

Exam-style Numerical Problem



Paths:

$$S \rightarrow A \rightarrow G : \quad 2 + 6 = 8$$

$$S \rightarrow B \rightarrow G : \quad 5 + 1 = 6$$

UCS chooses minimum accumulated cost, so:

$$S \rightarrow B \rightarrow G$$

with total cost

$$6$$

Python Code

```
import heapq

graph = {
    'S': [( 'A', 2), ( 'B', 5)],
    'A': [( 'G', 6)],
    'B': [( 'G', 1)],
    'G': []
}

def ucs(start, goal):
    pq = [(0, start, [start])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node in visited:
            continue
        visited.add(node)

        print(node, "cost =", cost)

        if node == goal:
            return path, cost

        for nxt, edge_cost in graph[node]:
            heapq.heappush(
                pq,
                (cost + edge_cost, nxt, path + [nxt])
            )

    print(ucs('S', 'G'))
```

2 Informed / Heuristic Search

Informed search uses additional problem-specific information.

A heuristic function is:

$$h(n) = \text{estimated cost from } n \text{ to goal}$$

For a goal node:

$$h(G) = 0$$

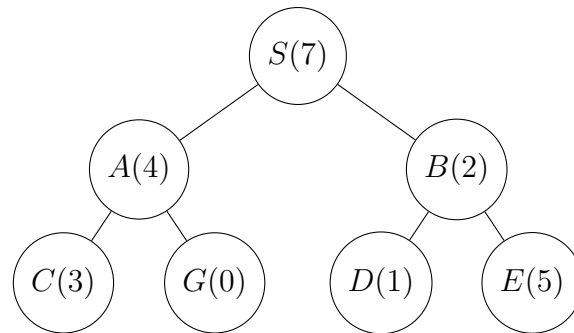
The provided previous-question material defines heuristic search as a strategy that uses heuristic information and distinguishes it from the heuristic function itself.

2.1 Greedy Best-First Search

Greedy Best-First Search uses:

$$f(n) = h(n)$$

extbfRule: choose the node with the smallest heuristic value.

Lab-Test-style Problem

Question: Starting from S , find G using Greedy Best-First Search.

Solution:

At S :

$A(4), B(2)$

Choose:

$B(2)$

Expand B :

$D(1), E(5)$

OPEN also contains $A(4)$. Therefore:

$D(1)$

is selected.

After D , the remaining best node is selected according to the OPEN heuristic values. Continue until $G(0)$ is selected.

Important: For Greedy Best-First Search, do not add $g(n)$ to the decision. Only $h(n)$ determines the priority.

Python Code

```

import heapq

graph = {
    'S': ['A', 'B'],
    'A': ['C', 'G'],
    'B': ['D', 'E'],
    'C': [], 'D': [], 'E': [], 'G': []
}

h = {
    'S': 7,
    'A': 4,
    'B': 2,
    'C': 3,
    'D': 1,
    'E': 5,
    'G': 0
}

```

```

def greedy_best_first(start, goal):
    pq = [(h[start], start, [start])]
    visited = set()

    while pq:
        hv, node, path = heapq.heappop(pq)
        print("Expand:", node, "h =", hv)

        if node == goal:
            return path

        if node in visited:
            continue
        visited.add(node)

        for child in graph[node]:
            if child not in visited:
                heapq.heappush(
                    pq, (h[child], child, path + [child])
                )

print(greedy_best_first('S', 'G'))

```

2.2 A* Search

A* combines path cost and heuristic estimate:

$$f(n) = g(n) + h(n)$$

where

$$g(n) = \text{cost from start to } n$$

and

$$h(n) = \text{estimated cost from } n \text{ to goal}$$

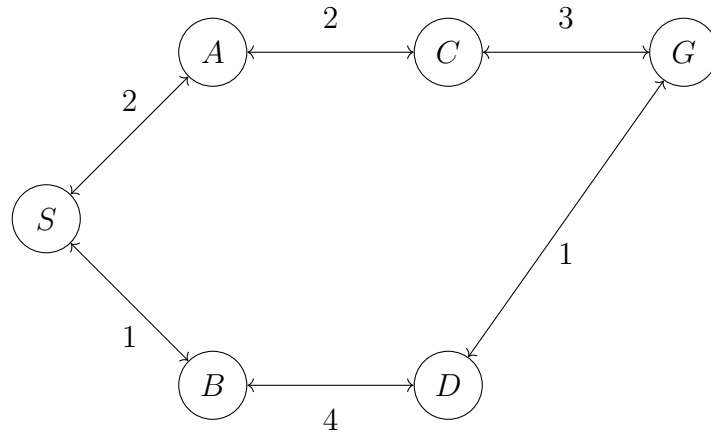
Thus A* balances:

cost already paid + estimated remaining cost.

The provided previous-question material states that A* is an informed search method and uses $f(n) = g(n) + h(n)$.

Exam-style A* Problem

Consider:



Heuristic values:

$$h(S) = 5, h(A) = 4, h(B) = 5, h(C) = 2, h(D) = 1, h(G) = 0.$$

Start:

$$g(S) = 0$$

$$f(S) = 0 + 5 = 5$$

Expand S .

For A :

$$g(A) = 2, \quad f(A) = 2 + 4 = 6$$

For B :

$$g(B) = 1, \quad f(B) = 1 + 5 = 6$$

There is a tie. Suppose A is selected first.

Expand A :

$$g(C) = 2 + 2 = 4$$

$$f(C) = 4 + 2 = 6$$

OPEN now contains $B(6)$ and $C(6)$. If C is selected:

$$g(G) = 4 + 3 = 7$$

$$f(G) = 7 + 0 = 7$$

But B remains with:

$$f(B) = 6.$$

Expand B :

$$g(D) = 1 + 4 = 5$$

$$f(D) = 5 + 1 = 6$$

Expand D :

$$g(G) = 5 + 1 = 6$$

which improves the previous goal cost of 7.

Therefore:

$$\boxed{S \rightarrow B \rightarrow D \rightarrow G}$$

Total cost:

$$\boxed{1 + 4 + 1 = 6}$$

Python Code

```
import heapq

graph = {
    'S': [('A', 2), ('B', 1)],
    'A': [('C', 2)],
    'B': [('D', 4)],
    'C': [('G', 3)],
    'D': [('G', 1)],
    'G': []
}

h = {
    'S': 5,
    'A': 4,
    'B': 5,
    'C': 2,
    'D': 1,
    'G': 0
}

def a_star(start, goal):
    pq = [(h[start], 0, start, [start])]
    best_g = {start: 0}

    while pq:
        f, g, node, path = heapq.heappop(pq)

        print("Expand:", node, "g =", g, "h =", h[node],
              "f =", f)

        if node == goal:
            return path, g

        if g != best_g.get(node, float("inf")):
            continue

        for child, cost in graph[node]:
            new_g = g + cost

            if new_g < best_g.get(child, float("inf")):
                best_g[child] = new_g
                new_f = new_g + h[child]
                heapq.heappush(
                    pq,
                    (new_f, new_g, child, path + [child])
                )

    print(a_star('S', 'G'))
```

3 Hill Climbing – Local Search and Optimization

Local search keeps track of a current state and moves to neighboring states. It does not need to keep the complete path.

Advantages:

- Very low memory use
- Useful for optimization
- Can work in very large or continuous spaces

3.1 Hill-Climbing Rule

For maximization:

Move to a better neighbor

For minimization:

Move to a lower-cost neighbor

The lecture describes hill climbing as continuously moving in the direction of increasing value and stopping at a peak.

3.2 Important Problems

- Local maximum
- Plateau
- Ridge

3.3 Mathematical Example

Consider:

$$f(x) = -(x - 5)^2 + 25$$

Possible neighbors:

$$x - 1, \quad x + 1$$

Start:

$$x = 1.$$

At $x = 1$:

$$f(1) = -(1 - 5)^2 + 25 = 9.$$

Neighbors:

$$f(0) = 0$$

$$f(2) = 16.$$

Since $16 > 9$, move:

$$1 \rightarrow 2.$$

At $x = 2$:

$$f(2) = 16.$$

The better neighbor is $x = 3$:

$$f(3) = 21.$$

At $x = 3$:

$$f(3) = 21, \quad f(4) = 24.$$

Move to 4.

At $x = 4$:

$$f(4) = 24, \quad f(5) = 25.$$

Move to 5.

At $x = 5$:

$$f(5) = 25.$$

The other neighbor:

$$f(6) = 24.$$

No neighbor is better.

Therefore:

$$x = 5, \quad f(x) = 25$$

Path:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$$

This follows the supplied local-search/CT material.

Python Code

```
def f(x):
    return -(x - 5)**2 + 25

def hill_climbing(start):
    current = start

    while True:
        left = current - 1
        right = current + 1

        best = current

        if f(left) > f(best):
            best = left

        if f(right) > f(best):
            best = right

        print("Current:", current, "Value:", f(current))

        if best == current:
            return current, f(current)

        current = best

print(hill_climbing(1))
```

3.4 Random-Restart Hill Climbing

If one hill-climbing run succeeds with probability p :

$$E(\text{runs}) = \frac{1}{p}$$

If a successful run requires s steps and a failed run requires f steps:

$$E(\text{steps}) = s + \left(\frac{1}{p} - 1\right) f$$

Numerical Example

Suppose:

$$p = 0.20, \quad s = 5, \quad f = 4.$$

Expected runs:

$$\frac{1}{0.20} = 5.$$

Expected steps:

$$\begin{aligned} 5 + \left(\frac{1}{0.20} - 1\right) 4 \\ = 5 + (5 - 1)4 \\ = 5 + 16 \end{aligned}$$

$$\boxed{21}$$

4 Gradient Descent

Gradient descent minimizes a cost function by moving in the negative gradient direction.

For one variable:

$$x_{\text{new}} = x_{\text{old}} - \eta \frac{dC}{dx}$$

For several variables:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} - \eta \nabla C(\mathbf{x})$$

where η is the learning rate.

4.1 Mathematical Example

Given:

$$f(x) = (x - 3)^2$$

Start:

$$x_0 = 0, \quad \eta = 0.1.$$

Derivative:

$$f'(x) = 2(x - 3).$$

Iteration 1:

$$f'(0) = -6.$$

$$x_1 = 0 - (0.1)(-6) = 0.6.$$

Iteration 2:

$$f'(0.6) = 2(0.6 - 3) = -4.8.$$

$$x_2 = 0.6 - (0.1)(-4.8) = 1.08.$$

Iteration 3:

$$f'(1.08) = 2(1.08 - 3) = -3.84.$$

$$x_3 = 1.08 - (0.1)(-3.84) = 1.464.$$

The values move toward:

$$\boxed{x = 3}$$

where

$$f(3) = 0.$$

Python Code

```
def f(x):
    return (x - 3)**2

def gradient(x):
    return 2 * (x - 3)

x = 0.0
eta = 0.1

for i in range(10):
    x = x - eta * gradient(x)
    print(i + 1, "x =", x, "f(x) =", f(x))
```

4.2 Gradient Descent – Lab Example

The lab code uses:

$$f(x) = x^2, \quad f'(x) = 2x,$$

with

$$x_0 = 5, \quad \eta = 0.1.$$

The update rule is:

$$\boxed{x_{new} = x - \eta(2x)}.$$

First iteration:

$$x_1 = 5 - (0.1)(2 \times 5) = 5 - 1 = \boxed{4}.$$

Second iteration:

$$x_2 = 4 - (0.1)(2 \times 4) = 4 - 0.8 = \boxed{3.2}.$$

Third iteration:

$$x_3 = 3.2 - (0.1)(2 \times 3.2) = 3.2 - 0.64 = \boxed{2.56}.$$

The value moves toward the minimum:

$$\boxed{x = 0, \quad f(x) = 0}.$$

5 Genetic Algorithm

A Genetic Algorithm is an evolutionary optimization/search method.

Main steps:

1. Initialization
2. Fitness evaluation
3. Selection
4. Crossover/recombination
5. Mutation
6. Replacement
7. Termination

5.1 Fitness-proportionate Selection

If fitness values are:

$$10, \quad 20, \quad 30$$

Total:

$$60.$$

Selection probabilities:

$$P_1 = \frac{10}{60} = 0.1667$$

$$P_2 = \frac{20}{60} = 0.3333$$

$$P_3 = \frac{30}{60} = 0.5.$$

Thus:

$$P_1 = 16.67\%, \quad P_2 = 33.33\%, \quad P_3 = 50\%$$

5.2 Crossover Example

Parents:

$$P_1 = 101101$$

$$P_2 = 111010$$

Cut after the third bit:

$$101|101$$

$$111|010$$

Children:

$$101010$$

$$111101$$

5.3 Mutation Example

Original:

110011

If the third bit is flipped:

111011

Python Code

```
import random

def fitness(chromosome):
    return sum(chromosome)

population = [
    [1, 0, 1, 1, 0],
    [1, 1, 1, 0, 0],
    [1, 1, 1, 1, 0],
    [0, 1, 0, 1, 1]
]

print("Initial population:")
for p in population:
    print(p, "fitness =", fitness(p))

def crossover(p1, p2, point=2):
    c1 = p1[:point] + p2[point:]
    c2 = p2[:point] + p1[point:]
    return c1, c2

def mutate(child, rate=0.1):
    child = child[:]
    for i in range(len(child)):
        if random.random() < rate:
            child[i] = 1 - child[i]
    return child

p1 = population[2]
p2 = population[1]

child1, child2 = crossover(p1, p2)

child1 = mutate(child1)
child2 = mutate(child2)

print("Children:", child1, child2)
```

6 Machine Learning

Machine learning allows a system to learn patterns from data.

For the present exam preparation, focus on:

- Classification
- Regression
- Confusion matrix
- Accuracy
- Precision
- Recall/Sensitivity
- Specificity
- F1-score
- MAE
- MSE
- RMSE
- R^2

The provided material identifies classification metrics including precision, recall, accuracy, F1-score and AUC, and regression metrics including RMSE and MAE.

6.1 Classification

Classification predicts a discrete class.

Examples:

- Spam / Not Spam
- Disease / No Disease
- Pass / Fail

6.2 Important Classification Algorithms from the Lab

The lab sheet highlights common classification algorithms: KNN, SVM, Naive Bayes and Random Forest.

K-Nearest Neighbors (KNN)

KNN classifies a new point using the classes of its k nearest training points. Usually, the majority class is selected.

Prediction = majority class among k nearest neighbors

Example: if $k = 3$ and the nearest classes are A, A, B , the prediction is $\boxed{A}$.

Support Vector Machine (SVM)

SVM finds a decision boundary (hyperplane) that separates classes while maximizing the margin. The closest points are called **support vectors**. A **kernel** can map data to a higher-dimensional space when a simple linear boundary is not enough.

The parameter C controls the penalty for classification errors: a larger C puts more penalty on misclassification.

Hyperplane = decision boundary

Margin = distance to support vectors

Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem and the assumption that features are conditionally independent given the class.

$$P(C | X) \propto P(C)P(X | C)$$

Random Forest

Random Forest combines many decision trees. For classification, the final class is normally selected by majority voting.

Random Forest = many decision trees + voting

Linear Regression vs Logistic Regression

Linear regression predicts a continuous numerical value. Logistic regression is used for classification and produces a probability between 0 and 1.

Continuous target $\rightarrow$ Linear Regression

Binary class $\rightarrow$ Logistic Regression

6.3 How a Machine Learning Model Works

For a typical supervised-learning task:

Data $\rightarrow$ Preprocessing $\rightarrow$ Training $\rightarrow$ Evaluation $\rightarrow$ Prediction

Short steps:

1. Collect labeled data.
2. Clean/preprocess the data and select useful features.
3. Choose a suitable model.
4. Train the model by learning parameters from training data.
5. Evaluate it on validation/test data using suitable metrics.
6. Use the trained model for prediction on unseen data.

6.4 Regression

Regression predicts a numerical value.

Examples:

- House price
- Student score
- Remaining Useful Life

6.5 Confusion Matrix

For binary classification:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

6.6 Classification Metrics

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

$$Precision = \frac{TP}{TP + FP}$$

Precision answers:

Of all predicted positives, how many were actually positive?

Recall / Sensitivity

$$Recall = \frac{TP}{TP + FN}$$

Recall answers:

Of all actual positives, how many were correctly detected?

Specificity

$$Specificity = \frac{TN}{TN + FP}$$

F1-score

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

ROC and AUC

The ROC curve shows True Positive Rate against False Positive Rate for different classification thresholds.

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$

AUC is the area under the ROC curve and summarizes class-separation ability across thresholds.

6.7 Classification Numerical Problem

Suppose a classifier produces:

$$TP = 80, \quad TN = 90, \quad FP = 10, \quad FN = 20.$$

Total:

$$80 + 90 + 10 + 20 = 200.$$

Accuracy:

$$\frac{80 + 90}{200} = \boxed{0.85 = 85\%}$$

Precision:

$$\frac{80}{80 + 10} = \frac{80}{90} = \boxed{0.8889}$$

Recall:

$$\frac{80}{80 + 20} = \boxed{0.80}$$

Specificity:

$$\frac{90}{90 + 10} = \boxed{0.90}$$

F1:

$$2 \frac{0.8889(0.80)}{0.8889 + 0.80} \approx \boxed{0.8421}$$

Python Code

```
TP, TN, FP, FN = 80, 90, 10, 20

accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)
f1 = 2 * precision * recall / (precision + recall)

print("Accuracy =", accuracy)
print("Precision =", precision)
print("Recall =", recall)
print("Specificity =", specificity)
print("F1 =", f1)
```

6.8 Regression Metrics

Given actual values:

$$y = [3, 5, 7, 9]$$

and predictions:

$$\hat{y} = [2, 6, 8, 8].$$

Errors:

$$e = y - \hat{y} = [1, -1, -1, 1].$$

MAE

$$MAE = \frac{|1| + |-1| + |-1| + |1|}{4}$$

$$\boxed{MAE = 1}$$

MSE

$$MSE = \frac{1^2 + (-1)^2 + (-1)^2 + 1^2}{4}$$

$$\boxed{MSE = 1}$$

RMSE

$$RMSE = \sqrt{MSE}$$

$$\boxed{RMSE = 1}$$

R^2

Mean actual value:

$$\bar{y} = \frac{3 + 5 + 7 + 9}{4} = 6.$$

Total sum of squares:

$$\begin{aligned} SS_{tot} &= (3 - 6)^2 + (5 - 6)^2 + (7 - 6)^2 + (9 - 6)^2 \\ &= 9 + 1 + 1 + 9 = 20. \end{aligned}$$

Residual sum of squares:

$$SS_{res} = 1 + 1 + 1 + 1 = 4.$$

Therefore:

$$R^2 = 1 - \frac{4}{20}$$

$$\boxed{R^2 = 0.8}$$

7 Neural Network – Lab Sheet Style

The Lab.pdf explains a basic neuron as

$$\boxed{z = wx + b}, \quad \boxed{\hat{y} = f(z)}.$$

Here w is the weight, x is the input, b is the bias and f is the activation function. The lab uses ReLU:

$$\boxed{ReLU(z) = \max(0, z)}.$$

7.1 Forward Pass

$$z = wx + b$$

$$\hat{y} = f(z)$$

So:

$$\boxed{x \rightarrow wx + b \rightarrow f(z) \rightarrow \hat{y}}$$

7.2 Loss Function

The lab uses squared-error loss:

$$\boxed{L = \frac{1}{2}(\hat{y} - y)^2}.$$

7.3 Backpropagation

Backpropagation calculates derivatives of the loss with respect to w and b using the chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial b}.$$

For ReLU:

$$\frac{d}{dz} ReLU(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}$$

and

$$\frac{\partial z}{\partial w} = x, \quad \frac{\partial z}{\partial b} = 1.$$

7.4 Complete Numerical Example from the Lab

Given:

$$w = 0.5, \quad b = -0.5, \quad x = 2,$$

and target:

$$y = x + 1 = 3.$$

1. Forward Pass

$$z = wx + b = (0.5)(2) - 0.5 = 0.5.$$

Since $z > 0$:

$$\hat{y} = ReLU(0.5) = \boxed{0.5}.$$

2. Loss

$$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(0.5 - 3)^2 = \boxed{3.125}.$$

3. Backpropagation

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 0.5 - 3 = -2.5.$$

Because $z = 0.5 > 0$:

$$\frac{\partial \hat{y}}{\partial z} = 1.$$

Thus:

$$\frac{\partial L}{\partial w} = (-2.5)(1)(2) = \boxed{-5}$$

$$\frac{\partial L}{\partial b} = (-2.5)(1)(1) = \boxed{-2.5}.$$

4. Gradient Descent Update

Using $\eta = 0.1$:

$$w_{new} = w - \eta \frac{\partial L}{\partial w} = 0.5 - (0.1)(-5) = \boxed{1.0}$$

$$b_{new} = b - \eta \frac{\partial L}{\partial b} = -0.5 - (0.1)(-2.5) = \boxed{-0.25}.$$

Therefore:

$$\boxed{w_{new} = 1.0, \quad b_{new} = -0.25}.$$

Important: Remember the complete flow:

Forward Pass $\rightarrow$ Loss

Backpropagation $\rightarrow$ Weight/Bias Update.

8 Lab Test #1 – Complete Solution**Question 1: Simple Linear Regression**

Given:

Student	Study Hours (x)	Score (y)
1	1	52
2	2	57
3	3	61
4	4	68
5	5	72

Mean

$$\bar{x} = \frac{1 + 2 + 3 + 4 + 5}{5} = 3$$

$$\bar{y} = \frac{52 + 57 + 61 + 68 + 72}{5} = 62$$

Slope

$$b_1 = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sum(x - \bar{x})^2}$$

x	y	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})(y - \bar{y})$	$(x - \bar{x})^2$
1	52	-2	-10	20	4
2	57	-1	-5	5	1
3	61	0	-1	0	0
4	68	1	6	6	1
5	72	2	10	20	4
				51	10

$$b_1 = \frac{51}{10} = \boxed{5.1}$$

Intercept

$$b_0 = \bar{y} - b_1\bar{x}$$

$$= 62 - (5.1)(3) = \boxed{46.7}$$

The question allows $0 \leq c \leq 1$. Choose:

$$\boxed{c = 0}.$$

Therefore:

$$\boxed{\hat{y} = 46.7 + 5.1x}$$

Predictions

$$\hat{y}_1 = 46.7 + 5.1(1) = 51.8$$

$$\hat{y}_2 = 56.9$$

$$\hat{y}_3 = 62.0$$

$$\hat{y}_4 = 67.1$$

$$\hat{y}_5 = 72.2$$

Student	Actual y	Predicted $\hat{y}$	Residual e
1	52	51.8	0.2
2	57	56.9	0.1
3	61	62.0	-1.0
4	68	67.1	0.9
5	72	72.2	-0.2

RMSE

$$\sum e^2 = 0.04 + 0.01 + 1 + 0.81 + 0.04 = 1.90.$$

$$RMSE = \sqrt{\frac{1.90}{5}}$$

$$RMSE \approx 0.616$$

R^2

$$\sum (y - \bar{y})^2 = 100 + 25 + 1 + 36 + 100 = 262.$$

$$R^2 = 1 - \frac{1.90}{262}$$

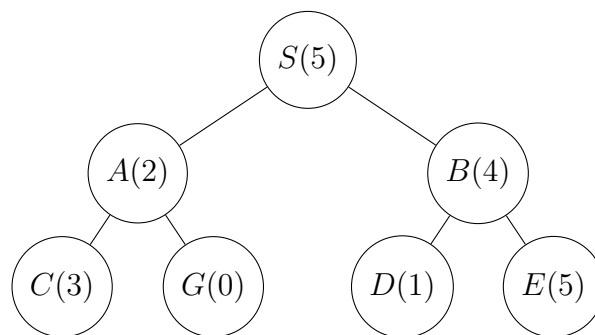
$$R^2 \approx 0.99275$$

or approximately:

$$99.275\%.$$

Question 2: Greedy Best-First Search

Given:



Greedy Best-First Search uses:

$$f(n) = h(n)$$

Start:

$$S(5)$$

Expand S :

$$A(2), B(4)$$

Choose:

$$A(2)$$

Expand A :

$$C(3), G(0)$$

The smallest heuristic is:

$$G(0)$$

Therefore:

$$\text{Goal Found}$$

Final path:

$$S \rightarrow A \rightarrow G$$

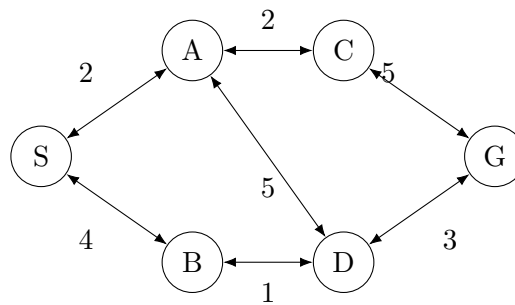
In location form:

$$\text{Home} \rightarrow A \rightarrow \text{Library}$$

9 AI Lab Assignments – Important Mathematical Problems

This section is based directly on the uploaded AI lab-assignment material and the separate MLP forward/backward worked example. Focus on hand calculation, intermediate steps, OPEN/CLOSED lists, gradients, and parameter updates.

9.1 Assignment 1 – A* Search: Route Finding



Question: Use A* graph search from S to G . Break ties alphabetically.

Edge	Cost
$S - A$	2
$S - B$	4
$A - C$	2
$A - D$	5
$B - D$	1
$C - G$	5
$D - G$	3

$$h(S) = 7, \quad h(A) = 6, \quad h(B) = 4, \quad h(C) = 4, \quad h(D) = 2, \quad h(G) = 0.$$

A* uses:

$$f(n) = g(n) + h(n)$$

Start:

$$g(S) = 0, \quad h(S) = 7, \quad f(S) = 7.$$

Expansion 1: S

$$A : \quad g = 2, \quad h = 6, \quad f = 8$$

$$B : \quad g = 4, \quad h = 4, \quad f = 8$$

Tie $\Rightarrow$ alphabetical order, so expand A .

Expansion 2: A

$$C : \quad g = 2 + 2 = 4, \quad h = 4, \quad f = 8$$

$$D : \quad g = 2 + 5 = 7, \quad h = 2, \quad f = 9$$

OPEN:

$$\boxed{B(8), C(8), D(9)}$$

Expand B .

Expansion 3: B

Through B :

$$g(D) = 4 + 1 = 5.$$

This improves the old value $g(D) = 7$, so update D :

$$D : \quad g = 5, \quad h = 2, \quad f = 7.$$

OPEN:

$$\boxed{D(7), C(8)}$$

Expand D .

Expansion 4: D

$$g(G) = 5 + 3 = 8, \quad h(G) = 0, \quad f(G) = 8.$$

OPEN:

$$\boxed{C(8), G(8)}$$

Tie $\Rightarrow$ expand C first.

Expansion 5: C

A path through C gives:

$$g(G) = 4 + 5 = 9.$$

The current value is already:

$$g(G) = 8.$$

So the new path is rejected.

Now select:

$$\boxed{G(8)}.$$

Final expansion order:

$$\boxed{S \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow G}$$

Final path:

$$\boxed{S \rightarrow B \rightarrow D \rightarrow G}$$

Total cost:

$$\boxed{4 + 1 + 3 = 8}$$

Admissibility

True least costs to G :

$$d^*(G) = 0, \quad d^*(D) = 3, \quad d^*(C) = 5,$$

$$d^*(B) = 1 + 3 = 4, \quad d^*(A) = \min(2 + 5, 5 + 3) = 7,$$

$$d^*(S) = 8.$$

Compare:

$$7 \leq 8, \quad 6 \leq 7, \quad 4 \leq 4, \quad 4 \leq 5, \quad 2 \leq 3.$$

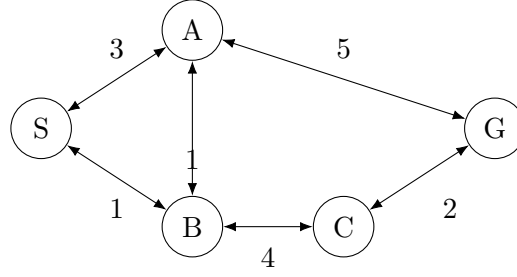
Therefore:

$$\boxed{\text{The heuristic is admissible.}}$$

A^* includes $g(n)$ because $h(n)$ alone ignores the cost already spent:

$$\boxed{f(n) = g(n) + h(n)}.$$

9.2 Assignment 2 – A* Search: Duplicate Paths and Consistency



Edges:

$$S - A = 3, \quad S - B = 1, \quad B - A = 1,$$

$$A - G = 5, \quad B - C = 4, \quad C - G = 2.$$

Heuristic:

$$h(S) = 7, \quad h(A) = 4, \quad h(B) = 6, \quad h(C) = 2, \quad h(G) = 0.$$

From S :

$$A : g = 3, \quad f = 3 + 4 = 7$$

$$B : g = 1, \quad f = 1 + 6 = 7.$$

Tie $\Rightarrow A$ first.

From A :

$$G : g = 3 + 5 = 8, \quad f = 8.$$

Then B is selected.

From B :

$$A : g = 1 + 1 = 2, \quad f = 2 + 4 = 6.$$

Thus:

$$\boxed{g(A) : 3 \rightarrow 2}$$

is an improved path and A is updated.

Also:

$$C : g = 1 + 4 = 5, \quad f = 5 + 2 = 7.$$

Because the graph is bidirectional and the heuristic is inconsistent, when the better value $g(A) = 2$ is found after A was expanded, A must be reopened (or allowed back into OPEN) before continuing A* graph search.

Reopen A :

$$G : g = 2 + 5 = 7, \quad f = 7.$$

Therefore:

$$S \rightarrow B \rightarrow A \rightarrow G$$

with total cost:

$$1 + 1 + 5 = 7.$$

Consistency Check

Consistency requires:

$$h(n) \leq c(n, n') + h(n').$$

For $S \rightarrow A$:

$$7 \leq 3 + 4 = 7 \quad \checkmark$$

For $S \rightarrow B$:

$$7 \leq 1 + 6 = 7 \quad \checkmark$$

For the undirected edge $A \leftrightarrow B$, both directions can be checked. The violating direction is $B \rightarrow A$:

$$6 \leq 1 + 4 = 5 \quad \times$$

Therefore:

The heuristic is not consistent.

Admissible:

$$h(n) \leq h^*(n)$$

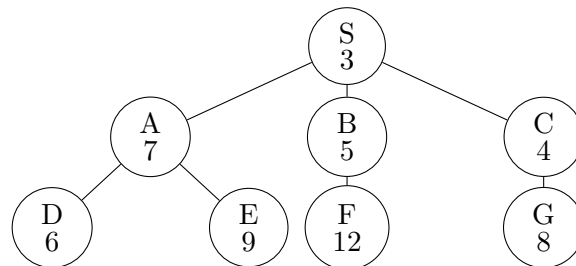
means the heuristic does not overestimate the true remaining cost.

Consistent:

$$h(n) \leq c(n, n') + h(n')$$

is the local edge-based condition.

9.3 Assignment 3 – Hill Climbing: Local Maximum



Values:

$$S = 3, A = 7, B = 5, C = 4, D = 6, E = 9, F = 12, G = 8.$$

Start at S using steepest ascent.

At S :

$$A = 7, \quad B = 5, \quad C = 4.$$

Choose A :

$$S \rightarrow A.$$

At A :

$$D = 6, \quad E = 9.$$

Choose E :

$$A \rightarrow E.$$

No better neighbor exists, so:

$$\boxed{S \rightarrow A \rightarrow E}$$

and the algorithm stops at:

$$\boxed{E, \text{ value} = 9}.$$

But:

$$F = 12 > 9.$$

Hence E is a:

$$\boxed{\text{local maximum, not global maximum.}}$$

Global maximum:

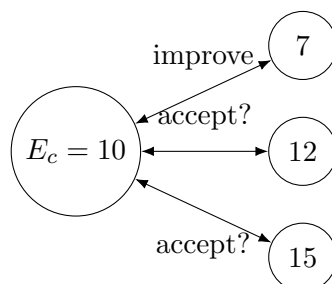
$$\boxed{F = 12}.$$

The algorithm fails because it only examines local neighbors and cannot cross a worse state to reach a better region.

A useful modification is:

$$\boxed{\text{Random-restart hill climbing}}.$$

9.4 Assignment 4 – Simulated Annealing



For minimization:

$$\Delta E = E_{\text{current}} - E_{\text{new}}.$$

If $\Delta E > 0$, accept automatically.

If $\Delta E \leq 0$:

$$p = e^{\Delta E/T}$$

and accept when:

$$r < p.$$

Trial	ΔE	T	p	Decision
(i)	3	5	1	Accept
(ii)	-2	10	0.8187	Accept
(iii)	-2	2	0.3679	Reject
(iv)	-5	20	0.7788	Accept

For (ii):

$$0.75 < 0.8187 \Rightarrow \text{Accept}.$$

For (iii):

$$0.75 > 0.3679 \Rightarrow \text{Reject}.$$

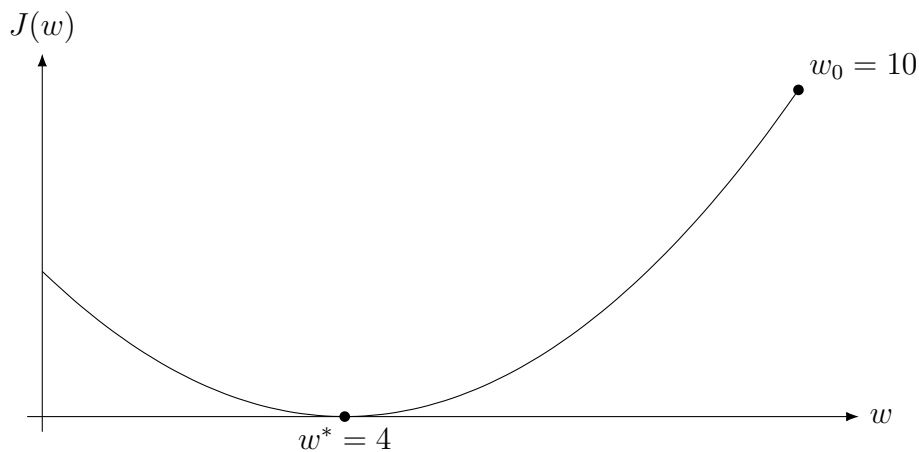
Higher T gives a higher probability of accepting a worse move.
As:

$$T \rightarrow 0,$$

worse moves become almost impossible, so simulated annealing becomes approximately:

$$\text{Hill climbing}.$$

9.5 Assignment 5 – Gradient Descent: One Variable



Minimize:

$$J(w) = (w - 4)^2$$

with:

$$w_0 = 10, \quad \eta = 0.25.$$

Derivative:

$$\boxed{\frac{dJ}{dw} = 2(w - 4)}.$$

Update:

$$w_{\text{new}} = w - \eta \frac{dJ}{dw} = w - 0.25[2(w - 4)]$$

$$\boxed{w_{\text{new}} = 0.5w + 2}.$$

Therefore:

$$w_1 = 0.5(10) + 2 = \boxed{7}$$

$$w_2 = 0.5(7) + 2 = \boxed{5.5}$$

$$w_3 = 0.5(5.5) + 2 = \boxed{4.75}.$$

Objective values:

$$J(w_0) = 36, \quad J(w_1) = 9, \quad J(w_2) = 2.25, \quad J(w_3) = 0.5625.$$

Exact minimizer:

$$2(w - 4) = 0 \Rightarrow \boxed{w = 4}.$$

For $\eta = 1.2$:

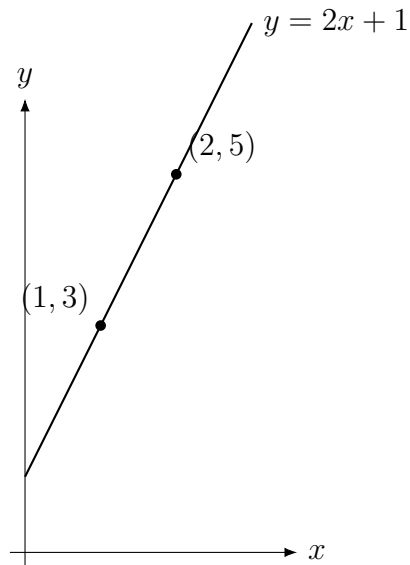
$$w_{\text{new}} = -1.4w + 9.6.$$

Starting from 10:

$$w_1 = -4.4, \quad w_2 = 15.76.$$

The algorithm oscillates and diverges because the learning rate is too large.

9.6 Assignment 6 – Gradient Descent for Linear Regression



Training samples:

$$(1, 3), \quad (2, 5).$$

Model:

$$\hat{y} = wx + b.$$

Loss:

$$L = \frac{1}{2}[(\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2].$$

Initial:

$$w = 0, \quad b = 0, \quad \eta = 0.1.$$

Forward pass:

$$\hat{y}_1 = 0, \quad \hat{y}_2 = 0.$$

$$L = \frac{1}{2}[(-3)^2 + (-5)^2] = \boxed{17}.$$

Gradients:

$$\begin{aligned} \frac{\partial L}{\partial w} &= (\hat{y}_1 - y_1)x_1 + (\hat{y}_2 - y_2)x_2 \\ &= (-3)(1) + (-5)(2) = \boxed{-13}. \end{aligned}$$

$$\frac{\partial L}{\partial b} = (-3) + (-5) = \boxed{-8}.$$

Updates:

$$w = 0 - 0.1(-13) = \boxed{1.3}$$

$$b = 0 - 0.1(-8) = \boxed{0.8}.$$

Second forward pass:

$$\hat{y}_1 = 1.3(1) + 0.8 = \boxed{2.1}$$

$$\hat{y}_2 = 1.3(2) + 0.8 = \boxed{3.4}.$$

New loss:

$$L = \frac{1}{2}[(2.1 - 3)^2 + (3.4 - 5)^2]$$

$$= \boxed{1.685}.$$

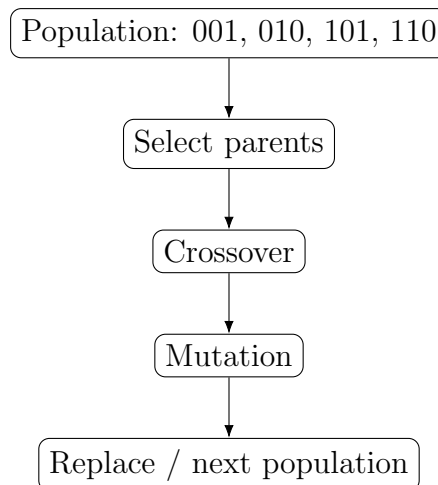
Exact parameters for:

$$y = 2x + 1$$

are:

$$\boxed{w = 2, \quad b = 1}.$$

9.7 Assignment 7 – Genetic Algorithm: Binary Maximization



Maximize:

$$f(x) = x(7 - x)$$

Initial population:

$$\{001, 010, 101, 110\}.$$

Chromosome	x	Fitness
001	1	6
010	2	10
101	5	10
110	6	6

Parents:

$$\boxed{010, 101}.$$

Crossover after bit 1:

$$0|10, \quad 1|01$$

so:

$$Child_1 = 001, \quad Child_2 = 110.$$

Flip the middle bit of Child 1:

$$001 \rightarrow \boxed{011}.$$

Fitness:

$$f(3) = 3(7 - 3) = \boxed{12}$$

and:

$$f(6) = \boxed{6}.$$

Assume an **elitist replacement** rule: keep the two best individuals from the combined parent/offspring pool. Then the best two are:

$$\boxed{011 (12)}$$

and one of the fitness-10 chromosomes:

$$\boxed{010 (10)} \quad \text{or} \quad \boxed{101 (10)}.$$

Global optima occur at:

$$f(3) = 12, \quad f(4) = 12.$$

Therefore:

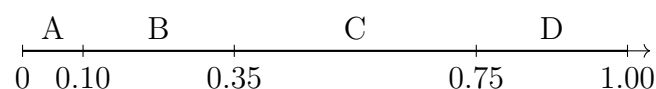
$$\boxed{x = 3, 4}$$

or:

$$\boxed{011, 100}.$$

Mutation is useful because it introduces new genetic variation and can help avoid premature convergence.

9.8 Assignment 8 – Genetic Algorithm: Selection Probabilities



Fitness:

$$A = 2, \quad B = 5, \quad C = 8, \quad D = 5.$$

Total:

$$20.$$

Probabilities:

$$P(A) = 0.10, \quad P(B) = 0.25, \quad P(C) = 0.40, \quad P(D) = 0.25.$$

Chromosome	Probability	Cumulative interval
A	0.10	[0, 0.10)
B	0.25	[0.10, 0.35)
C	0.40	[0.35, 0.75)
D	0.25	[0.75, 1.00)

For:

$$r = 0.05, 0.31, 0.74, 0.96$$

we obtain:

$$0.05 \rightarrow A, \quad 0.31 \rightarrow B, \quad 0.74 \rightarrow C, \quad 0.96 \rightarrow D.$$

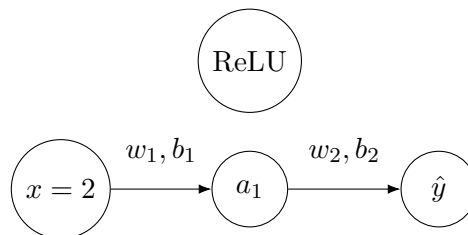
Thus:

$$\boxed{\text{Mating pool} = \{A, B, C, D\}}.$$

Because selection is with replacement, a chromosome may be selected more than once.
If all fitness values are equal:

$$\boxed{P(A) = P(B) = P(C) = P(D) = 0.25}.$$

9.9 Assignment 9 – MLP Forward Pass with ReLU



Given:

$$x = 2, \quad y = 5,$$

$$w_1 = 1.5, \quad b_1 = -1, \quad w_2 = 2, \quad b_2 = 0.5.$$

Hidden layer:

$$z_1 = w_1 x + b_1$$

$$a_1 = \text{ReLU}(z_1)$$

Output:

$$\hat{y} = w_2 a_1 + b_2.$$

Loss:

$$L = \frac{1}{2}(\hat{y} - y)^2.$$

Calculate:

$$z_1 = (1.5)(2) - 1 = \boxed{2}.$$

$$a_1 = \text{ReLU}(2) = \boxed{2}.$$

$$\hat{y} = (2)(2) + 0.5 = \boxed{4.5}.$$

$$L = \frac{1}{2}(4.5 - 5)^2 = \boxed{0.125}.$$

ReLU:

$$\boxed{\text{ReLU}(z) = \max(0, z)}$$

and:

$$\boxed{\text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}}$$

If $b_1 = -4$:

$$z_1 = (1.5)(2) - 4 = -1$$

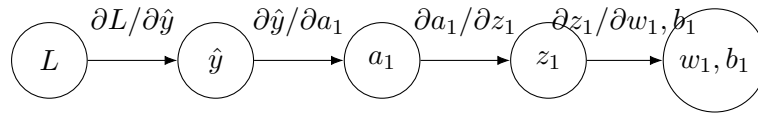
$$a_1 = 0$$

$$\hat{y} = (2)(0) + 0.5 = \boxed{0.5}$$

$$L = \frac{1}{2}(0.5 - 5)^2 = \boxed{10.125}.$$

When $z_1 < 0$, the ReLU derivative is zero. This is the basic dead-ReLU situation for that training sample.

9.10 Assignment 10 – MLP Backward Pass with ReLU



Use the same parameters:

$$x = 2, \quad y = 5, \quad w_1 = 1.5, \quad b_1 = -1, \quad w_2 = 2, \quad b_2 = 0.5$$

with:

$$\eta = 0.1.$$

Forward values:

$$z_1 = 2, \quad a_1 = 2, \quad \hat{y} = 4.5.$$

$$L = \frac{1}{2}(4.5 - 5)^2 = 0.125.$$

First:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = \boxed{-0.5}.$$

For the output neuron:

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_2} = (-0.5)(2) = \boxed{-1}$$

$$\frac{\partial L}{\partial b_2} = (-0.5)(1) = \boxed{-0.5}.$$

Backpropagate to the hidden neuron:

$$\frac{\partial L}{\partial a_1} = (-0.5)(2) = \boxed{-1}.$$

Since $z_1 > 0$:

$$\text{ReLU}'(z_1) = 1.$$

Therefore:

$$\frac{\partial L}{\partial z_1} = (-1)(1) = \boxed{-1}.$$

Now:

$$\frac{\partial L}{\partial w_1} = (-1)(2) = \boxed{-2}$$

and:

$$\frac{\partial L}{\partial b_1} = (-1)(1) = \boxed{-1}.$$

Parameter updates:

$$w_1^{new} = 1.5 - 0.1(-2) = \boxed{1.7}$$

$$b_1^{new} = -1 - 0.1(-1) = \boxed{-0.9}$$

$$w_2^{new} = 2 - 0.1(-1) = \boxed{2.1}$$

$$b_2^{new} = 0.5 - 0.1(-0.5) = \boxed{0.55}.$$

New forward pass:

$$z_1^{new} = (1.7)(2) - 0.9 = \boxed{2.5}$$

$$a_1^{new} = 2.5$$

$$\hat{y}^{new} = (2.1)(2.5) + 0.55 = \boxed{5.8}$$

$$L^{new} = \frac{1}{2}(5.8 - 5)^2 = \boxed{0.32}.$$

Here the loss increases from 0.125 to 0.32 after this particular update. This is not an arithmetic error; the chosen learning rate $\eta = 0.1$ is large enough to overshoot the minimum for this small example. A smaller learning rate would generally make the update more conservative.

The learning sequence is:

Forward $\rightarrow$ Loss $\rightarrow$ Backpropagation $\rightarrow$ Update $\rightarrow$ Forward again

9.11 Additional MLP Worked Example – $y = 2x + 1$

Given:

$$y = 2x + 1, \quad x = 2, \quad y = 5,$$

$$\hat{y} = \text{ReLU}(wx + b),$$

$$w = 1, \quad b = 0, \quad \eta = 0.1.$$

Forward pass:

$$z = wx + b = (1)(2) + 0 = \boxed{2}$$

$$\hat{y} = \text{ReLU}(2) = \boxed{2}$$

$$L = \frac{1}{2}(2 - 5)^2 = \boxed{4.5}.$$

Backward pass:

$$\frac{\partial L}{\partial \hat{y}} = 2 - 5 = \boxed{-3}.$$

Because $z > 0$:

$$\text{ReLU}'(z) = 1.$$

Therefore:

$$\frac{\partial L}{\partial z} = (-3)(1) = \boxed{-3}.$$

Then:

$$\frac{\partial L}{\partial w} = (-3)(2) = \boxed{-6}$$

$$\frac{\partial L}{\partial b} = (-3)(1) = \boxed{-3}.$$

Update:

$$w_{new} = 1 - 0.1(-6) = \boxed{1.6}$$

$$b_{new} = 0 - 0.1(-3) = \boxed{0.3}.$$

Second forward pass:

$$z_{new} = (1.6)(2) + 0.3 = \boxed{3.5}$$

$$\hat{y}_{new} = 3.5$$

$$L_{new} = \frac{1}{2}(3.5 - 5)^2 = \boxed{1.125}.$$

Thus:

$$\boxed{4.5 \rightarrow 1.125}$$

so the loss decreases.

For an active ReLU ($z > 0$):

$$\boxed{\frac{\partial L}{\partial w} = (\hat{y} - y)x}$$

$$\boxed{\frac{\partial L}{\partial b} = \hat{y} - y}$$

and therefore:

$$\boxed{w \leftarrow w - \eta(\hat{y} - y)x}$$

$$\boxed{b \leftarrow b - \eta(\hat{y} - y)}.$$

If $z < 0$, then:

$$\boxed{\text{ReLU}'(z) = 0}$$

and for that neuron and example:

$$\frac{\partial L}{\partial w} = 0, \quad \frac{\partial L}{\partial b} = 0.$$

The supplied worked example also notes that $y = 2x+1$ is represented exactly by $ReLU(2x+1)$ only where $2x+1 \geq 0$; a linear output is needed to represent the full real-valued line.

10 Quick Formula Sheet

Search

Romania Map:

$$\text{Greedy Best-First: } f(n) = h(n)$$

$$A^*: f(n) = g(n) + h(n)$$

For the Arad–Bucharest example:

$$\text{Greedy path cost} = 450$$

$$A^* \text{ path cost} = 418$$

$$BFS : \text{ shallowest first}$$

$$DFS : \text{ deepest first}$$

$$UCS : \min g(n)$$

$$\text{Greedy} : f(n) = h(n)$$

$$A^* : f(n) = g(n) + h(n)$$

$$IDS : DFS \text{ with increasing depth limit}$$

Local Search

$$x_{\text{new}} = x_{\text{old}} - \eta \nabla C(x)$$

$$E(\text{runs}) = \frac{1}{p}$$

$$E(\text{steps}) = s + \left(\frac{1}{p} - 1 \right) f$$

Classification

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$Specificity = \frac{TN}{TN + FP}$$

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

Regression

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE}$$

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Neural Network

$$z = \sum_i w_i x_i + b$$

$$a = \phi(z)$$

Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

ReLU:

$$ReLU(z) = \max(0, z)$$